

SERVICIOS TELEMÁTICOS

PROYECTO 1

BALANCEADOR DE CARGA WEB CON APACHE MOD_PROXY_BALANCER

INTEGRANTES

Isabella Ortiz Hernández

Julián Viafara

Samuel Sepúlveda

Sebastián Cobos

Isabela Cabezas

Valentina Velastegui

INTRODUCCIÓN

La creciente demanda de servicios web, impulsada por el comercio electrónico y la digitalización, expone las limitaciones de los servidores monolíticos.

✓ Problema

El aumento masivo de usuarios concurrentes satura los servidores únicos tradicionales, provocando alta latencia, caídas del sistema y una nula capacidad de respuesta ante fallos.

✓ Solución

Implementar el balanceo de carga como técnica fundamental para distribuir el tráfico equitativamente entre múltiples nodos, garantizando alta disponibilidad y optimización de recursos.

✓ Nuestro enfoque

Despliegue de tres servidores backend en contenedores Docker Compose, balanceados con el módulo Apache mod_proxy_balancer y validados bajo estrés extremo mediante pruebas con Artillery.

✓ Objetivo

Desarrollar un clúster web resiliente capaz de soportar picos de demanda simultánea sin interrupciones, evaluando el desempeño mediante latencia, throughput y tasa de error.

CONTEXTO

✓ El Impulso Digital :

- Digitalización acelerada de procesos cotidianos (compras, trabajo, estudios y trámites).
- Migración masiva de servicios empresariales hacia plataformas web.
- Efecto: Exigencia inmediata de disponibilidad continua y respuestas ultrarrápidas.

✓ El Desafío Técnico:

- Incremento crítico en el volumen de usuarios y peticiones concurrentes.
- Inoperancia de las arquitecturas tradicionales basadas en un único servidor para mitigar grandes cargas.

✓ Limitaciones e Impacto del Servidor Único:

- Sobrecarga y saturación crítica de los recursos de hardware.
- Aumento de la latencia (lentitud en el servicio).
- Puntos únicos de falla: Si el servidor único cae, todo el servicio se interrumpe y arruina la experiencia del usuario.



ALTERNATIVAS DE SOLUCIÓN

Teniendo en cuenta la problemática planteada se consideraron algunas alternativas de solución utilizando la metodología de balanceo de carga como:



Balanceador de carga web con NGINX que ofrece un manejo eficiente de conexiones con un bajo consumo de memoria, además proporciona una gran versatilidad ya que puede funcionar como servidor web, cache, balanceador y proxy inverso.



Balanceo de carga de carga web con HAProxy que es una solución especializada en balanceo de carga y alta disponibilidad. Destaca por su precisión en la distribución de tráfico, monitoreo de nodos backend y soporte para sesiones persistentes.



Balanceador de carga web con Apache mod_proxy_balancer esta fue la solución seleccionada para este proyecto, ya que nos permite crear una solución versátil, flexible, integrada y persistente. Debido a que soporta múltiples protocolos, también soporta sesiones basadas en cookies o codificación URL.

DISEÑO DE LA SOLUCIÓN

Se diseñó una arquitectura distribuida basada en contenedores Docker, utilizando Apache HTTP Server con mod_proxy_balancer para distribuir el tráfico y garantizar alta disponibilidad.

✓ Estructura en dos niveles

Balanceador frontend	Balanceador Backend
• Punto de entrada de las solicitudes HTTP. • Distribuye tráfico entre 2 instancias frontend desplegadas en Docker. • Mejora disponibilidad y elimina puntos únicos de fallo	• Funciona como API Gateway. • Redirige solicitudes hacia los microservicios según el endpoint solicitado. • Permite administración y escalabilidad independiente de servicios.

✓ Infraestructura backend

- Arquitectura basada en microservicios.
- Servicios desplegados en contenedores independientes.
- Base de datos centralizada para usuarios y publicaciones.

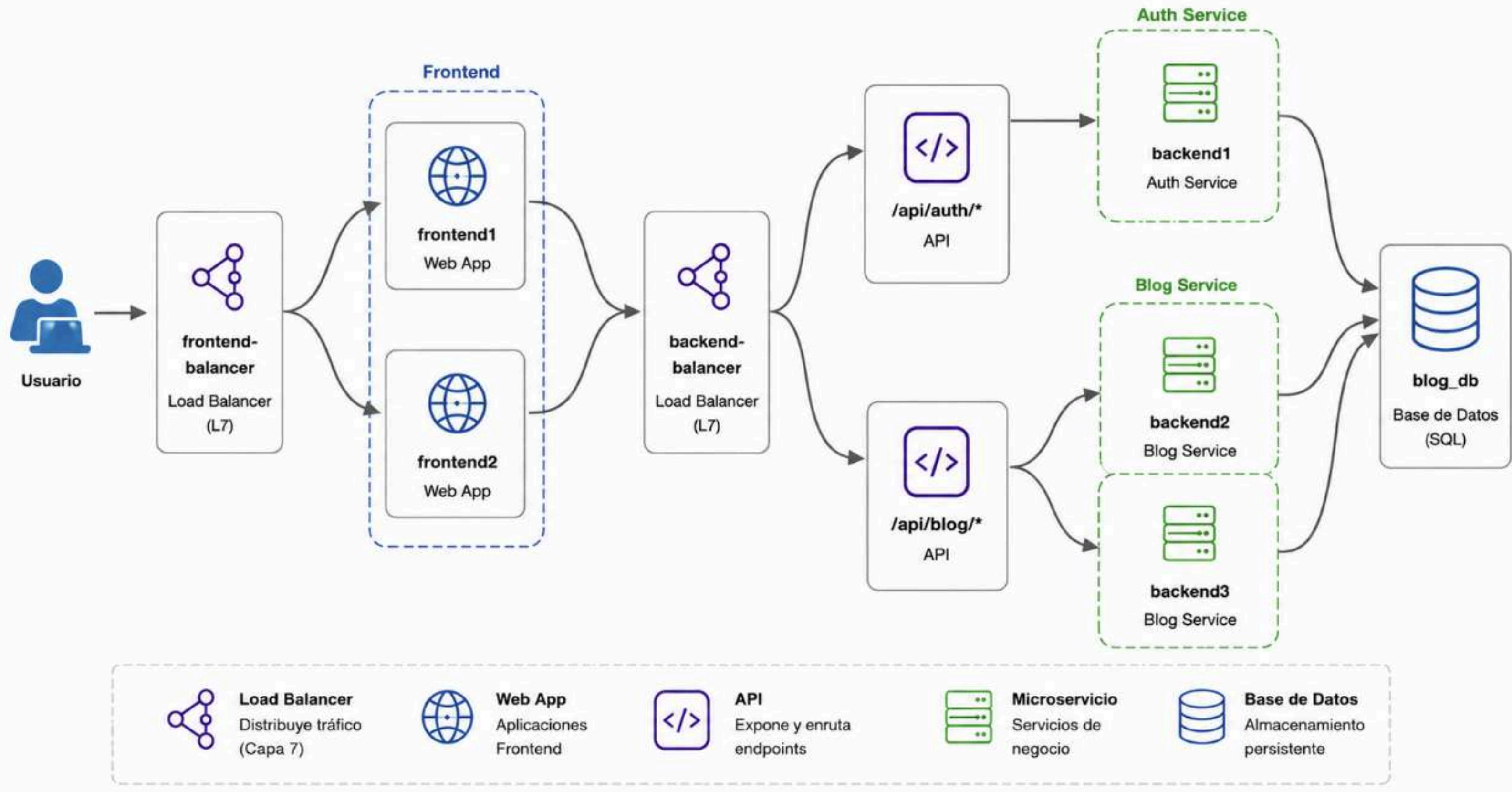
✓ Estrategias de balanceo

- Round Robin (byrequests): Distribución secuencial de solicitudes.
- Least Connections (bybusyness): Prioriza nodos con menor carga activa.

✓ Despliegue

Implementación automatizada con Docker y Docker Compose, utilizando redes internas y variables de entorno para facilitar escalabilidad horizontal.

DISEÑO DE LA SOLUCIÓN



IMPLEMENTACIÓN

1

Se utilizó Vagrant con VirtualBox para crear una máquina virtual servidor con Ubuntu 22.04, donde se despliega toda la infraestructura Dockerizada del proyecto.

```
Vagrantfile

Vagrant.configure("2") do |config|
  config.vm.boot_timeout = 600

  # SERVIDOR
  config.vm.define "servidor" do |servidor|
    servidor.vm.box = "ubuntu/jammy64"
    servidor.vm.hostname = "servidor"
    servidor.vm.network "private_network", ip: "192.168.50.3"
    servidor.vm.provider "virtualbox" do |vb|
      vb.memory = 2048
      vb.cpus = 2
    end
  end

  # CLIENTE
  config.vm.define "cliente" do |cliente|
    cliente.vm.box = "ubuntu/jammy64"
    cliente.vm.hostname = "cliente"
    cliente.vm.network "private_network", ip: "192.168.50.2"
    cliente.vm.provider "virtualbox" do |vb|
      vb.memory = 1024
      vb.cpus = 1
    end
  end
end
```

```
docker-compose.yml
1  services:
2    # =====
3    # ARTILLERY
4    # =====
5  > artillery: ...
13  # =====
14  # AUTH BACKEND
15  # =====
16  > backend1: ...
34  # =====
35  # BLOG BACKEND 1
36  # =====
37  > backend2: ...
53  # =====
54  # BLOG BACKEND 2
55  # =====
56  > backend3: ...
72  # =====
73  # BACKEND LOAD BALANCER
74  # =====
75  > backend-balancer: ...
86  # =====
```

```
docker-compose.yml X
1  services:
86  # =====
87  # FRONTEND INSTANCE 1
88  # =====
89  > frontend1: ...
98  # =====
99  # FRONTEND INSTANCE 2
100 # =====
101 > frontend2: ...
110 # =====
111 # FRONTEND LOAD BALANCER
112 # =====
113 > frontend-balancer: ...
123 # =====
124 # DATABASE
125 # =====
126 > mysql: ...
143 # =====
144 # NETWORK
145 # =====
146 > networks: ...
149
```

2

El archivo docker-compose.yml orquesta todos los servicios (balanceadores, backends, frontends, base de datos), configurando redes internas, variables de entorno y permitiendo el escalado horizontal.

IMPLEMENTACIÓN



Las instancias frontend desarrolladas con React + Vite se sirven en contenedores Docker independientes, balanceadas por Apache con mod_proxy_balancer usando algoritmo Round Robin.

```

1  # apache.conf  X
2
3  frontend-balancer > # apache.conf
4  #LoadModule ssl_module modules/mod_ssl.so
5
6  LoadModule authz_host_module modules/mod_authz_host.so
7  LoadModule dir_module modules/mod_dir.so
8  LoadModule mime_module modules/mod_mime.so
9  LoadModule alias_module modules/mod_alias.so
10 LoadModule unixd_module modules/mod_unixd.so
11 LoadModule log_config_module modules/mod_log_config.so
12
13 LoadModule proxy_module modules/mod_proxy.so
14 LoadModule proxy_balancer_module modules/mod_proxy_balancer.so
15 LoadModule proxy_http_module modules/mod_proxy_http.so
16 LoadModule slotmem_shm_module modules/mod_slotmem_shm.so
17 LoadModule lbmethod_byrequests_module modules/mod_lbmethod_byrequests.so
18
19 User daemon
20 Group daemon
21
22 Listen 80
23
24 ServerAdmin you@example.com
25 DocumentRoot "/usr/local/apache2/htdocs"
26
27 <Directory "/usr/local/apache2/htdocs">
28     AllowOverride None
29     Require all granted
30 </Directory>
31
32 <Proxy "balancer://frontendcluster">
33     BalancerMember http://frontend1:80 route=f1
34     BalancerMember http://frontend2:80 route=f2
35     ProxySet lbmethod=byrequests sticky=session=ROUTEID
36 </Proxy>
37
38 ProxyPass "/" "balancer://frontendcluster/"
39 ProxyPassReverse "/" "balancer://frontendcluster/"
40
41 FallBackResource /index.html
42
43 <Location "/balancer-manager">
44     SetHandler balancer-manager
45     Require all granted
46 </Location>

```

```

16 # Algoritmo 1: Round Robin
17 LoadModule lbmethod_byrequests_module modules/mod_lbmethod_byrequests.so
18
19 # Algoritmo 2: Least Connections
20 LoadModule lbmethod_bybusyness_module modules/mod_lbmethod_bybusyness.so
21
22
23 # =====
24 # BLOG_CLUSTER
25 # =====
26
27 <Proxy "balancer://blog_cluster">
28     BalancerMember http://backend2:3000
29     BalancerMember http://backend3:3000
30
31 #Descomentar para usar Least Connections
32 ProxySet lbmethod-bybusyness
33
34 #Descomentar para usar Least Round Robin
35 # ProxySet lbmethod-byrequests
36 </Proxy>
37
38 # =====
39 # AUTH_ROUTING
40 # =====
41
42 ProxyPass "/api/auth/" "http://backend1:3000/api/auth/"
43 ProxyPassReverse "/api/auth/" "http://backend1:3000/api/auth/"
44
45 # =====
46 # BLOG_ROUTING
47 # =====
48
49 ProxyPass "/api/blog" "balancer://blog_cluster/api/blog/"
50 ProxyPassReverse "/api/blog" "balancer://blog_cluster/api/blog/"
51
52 # =====
53 # BALANCER_MANAGER
54 # =====
55
56 <location "/balancer-manager">
57     SetHandler balancer-manager

```

4

Se utilizó Apache HTTP Server como puerta de enlace API para implementar el balanceador de backend. Este componente recibe las peticiones que llegan desde el frontend y las encamina de manera dinámica hacia los microservicios adecuados, según el endpoint requerido.

IMPLEMENTACIÓN

5

Evolución de arquitectura

- Inicialmente el sistema funcionaba bajo una arquitectura monolítica, concentrando todas las funcionalidades en un único backend.
- Para mejorar escalabilidad, mantenimiento y tolerancia a fallos, se migró hacia una arquitectura basada en microservicios.

Distribucion de Microservicios

Servicio	Funcionalidad
backend1	Autenticación y seguridad
backend2	Gestión de publicaciones
backend3	Réplica del servicio de publicaciones

Enrutamiento del API Gateway

Ruta	Destino
/api/auth/*	backend1
/api/blog/*	backend2, backend3

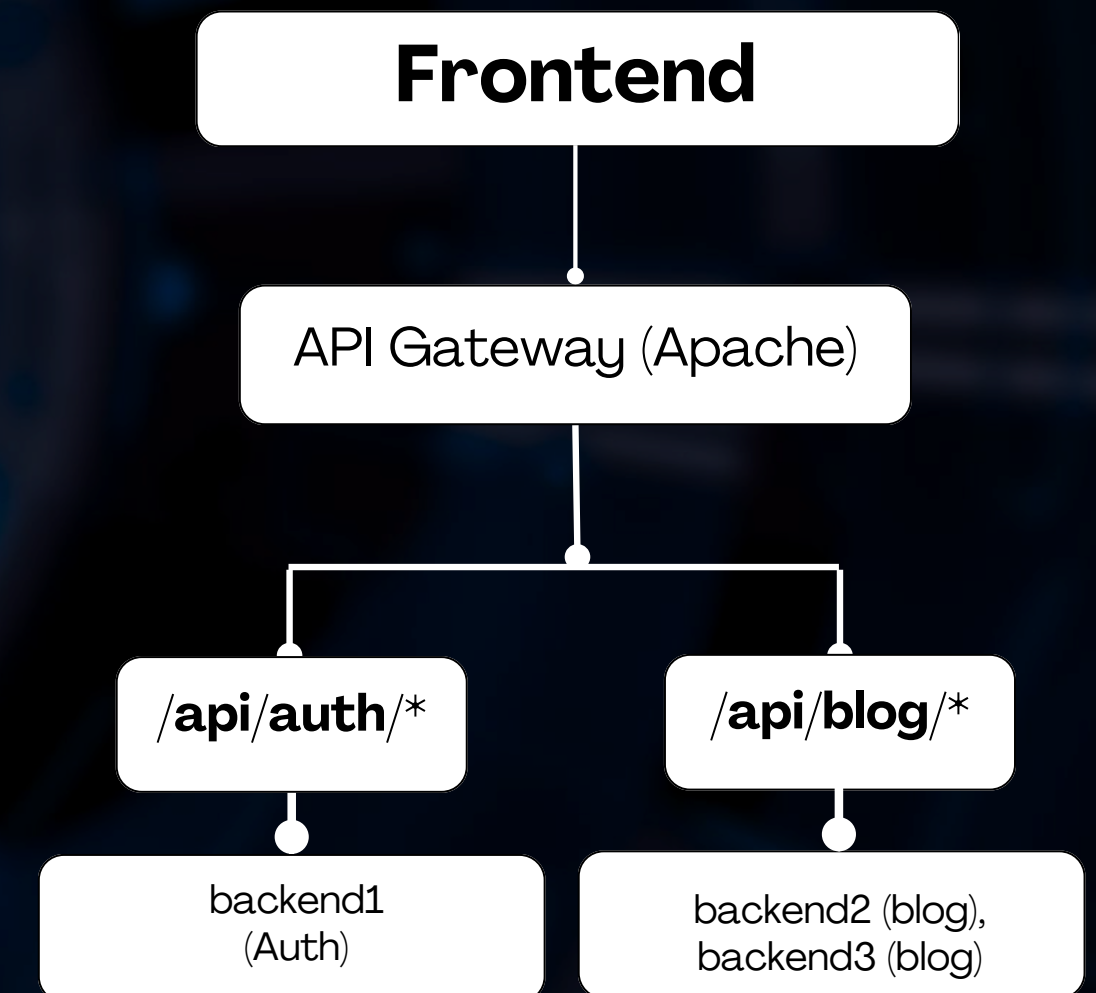
Funciones implementadas

backend1 (Auth Service):

- Registro de usuarios
- Inicio de sesión
- Generación de tokens JWT
- Cifrado con bcrypt

backend2 y backend3 (Blog Service):

- Gestión de contenido
- Publicación y consulta de blogs



VIDEO




```
vagrant@servidor: ~/PROYECT x  vagrant@servidor: ~/PROYEC x  +  v
frontend1-1 | 172.18.0.6 -- [18/May/2026:16:08:37 +0000] "GET /assets/index-BTOxXuQ9.js HTTP
/1.1" 404 555 "http://192.168.50.3:8090/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.3
6 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36" "192.168.50.1"
frontend1-1 | 172.18.0.6 -- [18/May/2026:16:08:37 +0000] "GET / HTTP/1.1" 200 504 "-" "Mozil
la/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/53
7.36" "192.168.50.1"
frontend1-1 | 172.18.0.6 -- [18/May/2026:16:08:37 +0000] "GET /assets/index-BJLs4Uci.css HTT
P/1.1" 200 8150 "http://192.168.50.3:8090/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537
.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36" "192.168.50.1"
frontend2-1 | 172.18.0.6 -- [18/May/2026:16:08:37 +0000] "GET /assets/index-Dq07EGSx.js HTTP
/1.1" 404 555 "http://192.168.50.3:8090/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.3
6 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36" "192.168.50.1"
frontend2-1 | 172.18.0.6 -- [18/May/2026:16:08:38 +0000] "GET / HTTP/1.1" 200 504 "-" "Mozil
la/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/53
7.36" "192.168.50.1"
frontend1-1 | 172.18.0.6 -- [18/May/2026:16:08:38 +0000] "GET /assets/index-BTOxXuQ9.js HTTP
/1.1" 404 555 "http://192.168.50.3:8090/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.3
6 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36" "192.168.50.1"
frontend2-1 | 172.18.0.6 -- [18/May/2026:16:08:38 +0000] "GET /assets/index-BJLs4Uci.css HTT
P/1.1" 200 8150 "http://192.168.50.3:8090/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537
.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36" "192.168.50.1"
frontend1-1 | 172.18.0.6 -- [18/May/2026:16:08:39 +0000] "GET / HTTP/1.1" 200 504 "-" "Mozil
la/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/53
7.36" "192.168.50.1"
frontend1-1 | 172.18.0.6 -- [18/May/2026:16:08:39 +0000] "GET /assets/index-BJLs4Uci.css HTT
P/1.1" 200 8150 "http://192.168.50.3:8090/" "Mozilla/5.0 (Windows N. 10.0; Win64; x64) AppleWebKit/537
.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36" "192.168.50.1"
frontend2-1 | 172.18.0.6 -- [18/May/2026:16:08:39 +0000] "GET /assets/index-Dq07EGSx.js HTTP
/1.1" 404 555 "http://192.168.50.3:8090/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.3
6 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36" "192.168.50.1"
backend-balancer-1 | [Mon May 18 16:09:44.816162 2026] [mpm_event:notice] [pid 1:tid 1] AH00492: cau
ght SIGWINCH, shutting down gracefully
frontend-balancer-1 exited with code 0
backend-balancer-1 exited with code 0
frontend2-1 | 2026/05/18 16:09:46 [notice] 1#1: signal 3 (SIGQUIT) received, shutting down
frontend2-1 | 2026/05/18 16:09:46 [notice] 30#30: gracefully shutting down
frontend2-1 | 2026/05/18 16:09:46 [notice] 30#30: exiting
frontend2-1 | 2026/05/18 16:09:46 [notice] 30#30: exit
frontend2-1 | 2026/05/18 16:09:46 [notice] 29#29: gracefully shutting down
frontend2-1 | 2026/05/18 16:09:46 [notice] 29#29: exiting
frontend2-1 | 2026/05/18 16:09:46 [notice] 29#29: exit
frontend1-1 | 2026/05/18 16:09:46 [notice] 1#1: signal 3 (SIGQUIT) received, shutting down
frontend1-1 | 2026/05/18 16:09:46 [notice] 30#30: gracefully shutting down
frontend1-1 | 2026/05/18 16:09:46 [notice] 30#30: exiting
frontend1-1 | 2026/05/18 16:09:46 [notice] 30#30: exit
frontend1-1 | 2026/05/18 16:09:46 [notice] 29#29: gracefully shutting down
frontend1-1 | 2026/05/18 16:09:46 [notice] 29#29: exiting
frontend1-1 | 2026/05/18 16:09:46 [notice] 29#29: exit
frontend2-1 | 2026/05/18 16:09:46 [notice] 1#1: signal 17 (SIGCHLD) received from 30
frontend2-1 | 2026/05/18 16:09:46 [notice] 1#1: worker process 29 exited with code 0
frontend2-1 | 2026/05/18 16:09:46 [notice] 1#1: worker process 30 exited with code 0
```

DevTools is now available in Spanish Don't show again Always match Chrome's language Switch DevTools to Spanish

Elements Console Sources **Network** Performance Memory Application Security Lighthouse

Preserve log Disable cache No throttling

Filter ☐ Invert More filters All Fetch/XHR Doc CSS JS Font Img Media Manifest Socket Wasm Other

Name	Status	Type	Initiator	Size	Time
login	200	document	Other	0.8 kB	6 m
index-Dq07EGSx.js	200	script	login.7	218 kB	21 m
index-BJLs4Uci.css	200	stylesheet	login.8	8.4 kB	8 m
css2?family=Interwght@400;500;600;700;800&family=JetB...	200	stylesheet	index-BJLs4Uci.css.1	1.0 kB	85 m
UcC73FwrK3iLTeHuS_nVMrMxCp50Sjla1ZL7.woff2	200	font	fonts.googleapis.com/css2?family=	48.3 kB	30 m
tDbv2o-fIEEnyOFZhsfKu5WU4zr3E_8X0PnT8RD8yKw8Nntka...	200	font	fonts.googleapis.com/css2?family=	31.5 kB	28 m
login	200	document	Other	0.8 kB	5 m
index-BTOxXuQ9.js	200	script	login.7	218 kB	13 m
index-BJLs4Uci.css	200	stylesheet	login.8	8.4 kB	8 m
css2?family=Interwght@400;500;600;700;800&family=JetB...	200	stylesheet	index-BJLs4Uci.css.1	1.0 kB	72 m
UcC73FwrK3iLTeHuS_nVMrMxCp50Sjla1ZL7.woff2	200	font	css2?family=Interwght@400-500	48.3 kB	25 m
tDbv2o-fIEEnyOFZhsfKu5WU4zr3E_8X0PnT8RD8yKw8Nntka...	200	font	css2?family=Interwght@400-500	31.5 kB	24 m

12 requests 617 kB transferred 653 kB resources Finish: 2.73 s DOMContentLoaded: 2.69 s Load: 2.74 s

Video de pruebas

V

VALENTINA VELASTEGUI LOPEZ

Balanceador de carga con Apache usando mod_proxy_balancer (Video de pruebas)

58/seg

908

http.request_rate

http.response_time

min: 0

max: 11

mean: 2

median: 2

p95: 5

p99: 7.9

http.response_time.2xx

min: 0

max: 11

mean: 2

median: 2

p95: 5

p99: 7.9

http.responses

users.created

users.created_by_name

Prueba 500 usuarios

145

Metrics for period to: 22:43:00(+0000) (width: 9.99%)

custom.distribution.unknown

2794

http.codes.200

2794

http.downloaded_bytes

2468176

http.request_rate

280/sec

http.requests

2795

http.response_time

min: 0

max: 41

mean: 8.2

median: 3

p95: 13.1

p99: 19.9

http.response_time.2xx

min: 0

max: 41

mean: 8.2

median: 3

p95: 13.1

p99: 19.9

http.responses

2794

users.created

145

users.created_by_name

Prueba 500 usuarios

145

Watch on  YouTube

PRUEBAS

Se realizaron pruebas de rendimiento y funcionalidad en los balanceadores frontend y backend para verificar la arquitectura implementada. Estas permitieron comprobar el comportamiento del sistema ante la concurrencia, su tolerancia a fallos y la distribución del tráfico, utilizando Artillery en un contenedor Docker para las pruebas de carga.

1) La primera validación fue verificar si el balanceador frontend repartía las peticiones de manera adecuada entre las dos instancias de la aplicación web.

Frontend:

```
for 1 in {1..10}; do curl http://192.168.50.3:8090; echo ""; done
```

2) Posteriormente, se validó el funcionamiento del balanceador backend y el API Gateway encargado de enrutar solicitudes hacia los microservicios correspondientes.

Backend:

```
for 1 in {1..10}; do curl http://192.168.50.3:8080; echo ""; done
```

3) También se realizaron pruebas para validar el estado de disponibilidad de los servicios backend mediante endpoints de health check.

Health check:

```
for 1 in {1..10}; do curl http://192.168.50.3:8080/health; echo ""; done
```


PRUEBAS

4) Con el propósito de examinar cómo funciona el sistema en diferentes situaciones de concurrencia, se crearon varios scripts de pruebas con Artillery.

Archivo	Escenario
artillery/balanceador-50.yml	50 usuarios simultáneos
artillery/balanceador-100.yml	100 usuarios simultáneos
artillery/balanceador-500.yml	500 usuarios simultáneos
artillery/balanceador-1000.yml	1000 usuarios simultáneos
artillery/balanceador-5000.yml	5000 usuarios simultáneos
artillery/balanceador-failover.yml	Prueba de caída del backend

Se examinaron las siguientes métricas a lo largo de cada prueba:

Throughput: Número de solicitudes que se procesan por segundo. Tiempo promedio de respuesta.

p95 El tiempo de respuesta para el 95% de las solicitudes.

p99 Tiempo de respuesta del 99 % de las peticiones.

HTTP 200: Número de respuestas exitosas. Solicitudes fallidas y cantidad de solicitudes que no han tenido éxito

DISCUSIÓN DE LAS PRUEBAS

Los resultados de las pruebas de carga usando el algoritmo round robin se resumen en la siguiente tabla:

Prueba	Requests	HTTP 200	Completados	Fallidos	Req/s	Mean	p95	p99
50 usuarios	936	919	33	17	27/s	9.6 ms	32.8 ms	49.9 ms
100 usuarios	2.831	2.831	72	0	56/s	6.1 ms	18 ms	80.6 ms
500 usuarios	2.708	2.591	10	108	57/s	1044.7 ms	25.8 ms	50.9 ms
1000 usuarios	19.714	19.564	586	145	228/s	4066.3 ms	450.4 ms	2231 ms
5000 usuarios	25000	25000	5000	0	187/s	4.9 ms	18 ms	47.9 ms
Failover	1200	1200	1200	0	19/s	1.4 ms	3 ms	6 ms

DISCUSIÓN DE LAS PRUEBAS

Los resultados de las pruebas de carga usando el algoritmo least connections se resumen en la siguiente tabla:

Prueba	Requests	HTTP 200	VUsers completados	Fallidos	Req/s	Mean	p95	p99
50 usuarios	1500	1500	50	0	32/s	3.0 ms	7.9 ms	15 ms
100 usuarios	3000	3000	100	0	61/s	2.4 ms	6 ms	12.1 ms
500 usuarios	15000	15000	500	0	260/s	22.6 ms	98.5 ms	214.9 ms
1000 usuarios	29.932	29920	988	12	387/s	63.7 ms	202.4 ms	278.7 ms
5000 usuarios	25000	25000	5000	0	199/s	2.8 ms	10.1 ms	32.8 ms
Failover	1200	1200	1200	0	19/s	1.4 ms	3 ms	6 ms

DISCUSIÓN DE PRUEBAS

Comparación de algoritmos de balanceo:

Escenario	Round Robin (byrequests)	Least Connections (bybusyness)
50 usuarios	Funcionamiento estable, baja latencia (9.6 ms), aunque se registraron 17 fallos.	Funcionamiento estable, menor latencia (3.0 ms) y sin errores.
100 usuarios	Buen rendimiento, 56 req/s y sin solicitudes fallidas.	Mejor tiempo de respuesta (2.4 ms), mayor throughput y sin errores.
500 usuarios	Aumento significativo de latencia (>1000 ms) y 108 solicitudes fallidas.	Distribución eficiente de carga, 260 req/s y 0 errores.
1000 usuarios	Degradación notable del rendimiento, 145 fallos y latencia superior a 4000 ms.	Mejor escalabilidad, menor latencia (63.7 ms) y solo 12 fallos.
5000 usuarios	Todas las solicitudes completadas, comportamiento estable.	Todas las solicitudes completadas, mejor latencia y throughput.
Failover	Redirección exitosa del tráfico tras caída de un backend.	Mantiene disponibilidad y continuidad del servicio.

CONCLUSIONES

- ✓ La implementación de una arquitectura basada en balanceo de carga, contenedores y microservicios permitió mejorar significativamente la disponibilidad del sistema, reducir los puntos únicos de fallo y facilitar su escalabilidad horizontal, permitiendo una mejor distribución de las solicitudes en un entorno distribuido.
- ✓ El uso de Docker y Docker Compose facilitó la automatización del despliegue, la orquestación de servicios y la gestión independiente de cada componente, simplificando la administración de la infraestructura completa del sistema.
- ✓ La segmentación del backend en microservicios permitió separar responsabilidades funcionales críticas como autenticación y gestión de contenido, lo que mejoró la modularidad del sistema, su mantenibilidad y la capacidad de escalar componentes de forma independiente según la demanda.

CONCLUSIONES

- ✓ El análisis comparativo de los algoritmos de balanceo evidenció que Least Connections presenta un mejor desempeño en escenarios de alta concurrencia debido a su distribución dinámica de carga, mientras que Round Robin mantiene un comportamiento estable en cargas moderadas al distribuir las solicitudes de forma uniforme entre los nodos disponibles.
- ✓ Las pruebas de estrés y tolerancia a fallos demostraron que la arquitectura implementada mantiene la continuidad del servicio incluso bajo altos niveles de carga y ante la caída de nodos del backend, lo que valida su robustez, disponibilidad y capacidad de recuperación.
- ✓ En términos generales, la integración de balanceo de carga multinivel, arquitectura de microservicios y contenedorización permite construir sistemas distribuidos resilientes, capaces de adaptarse de manera eficiente a diferentes niveles de demanda y condiciones de operación.

The background is a deep blue with a subtle, abstract pattern of light blue lines and dots, resembling a circuit board or a digital network. A large, dark blue rectangle is centered horizontally, serving as a backdrop for the text. On the left and right sides of this rectangle, there are stylized white circuit traces that extend towards the edges of the frame. The word "GRACIAS" is written in a bold, white, sans-serif font, centered within the dark rectangle.

GRACIAS